

OPERATION IRON WEB: A Mesh Worm, Its Propagation, and Its Containment on an RP2350 Industrial Tamper System

Kevin Thomas
George Mason University
Fairfax, VA, USA
kthoma60@gmu.edu

Abstract: One clean node is not a clean network. The OPERATION IRON WEB build is a bare-metal RP2350 industrial tamper system and its companion mesh gateway, and it is Act V of the OPERATION COLD IRON story. A ring of chassis-intrusion nodes reads an SG90 servo as a shutter latch, a VS1838B infrared receiver as a local arm/disarm remote for ARM, DISARM, and CLEAR, a DHT11 as the cabinet temperature sensor, a 1602 LCD as the tamper state, zone, and infection readout, red/yellow/green LEDs as the INTRUSION, ARMED, and SECURE annunciator, a debounced button as the manual arm request, and an RYLR998 LoRa link to a sealed tamper gateway. Every request and command is sealed end to end with XChaCha20-Poly1305 (RFC 8439 ChaCha20 and Poly1305 with an HChaCha20 subkey) keyed through Argon2id (RFC 9106, profile t=3, p=1, m=64 blocks), implemented in-repo with no third-party code and tested against published vectors. Act V takes the persistence lesson of Act IV and makes it propagate: a benign FROSTLINE worm, compiled only under a `SANDBOX_ONLY` guard, listens on the raw inbound payload for the 7-byte magic `IRONWEB`, erases and programs a one-time `0xC7` infection marker into the reserved flash sector `0x103FF000`, re-installs on every later boot, and, every four ticks while infected and not under a probe, emits an 11-byte progress frame (the magic, the marker, the tick counter, and a debug flag) so a neighboring node that receives it infects itself and emits the frame again. It reads CoreDebug DHCSR at `0xE000EDF0` to suppress itself while a debug probe is attached. We document the peripheral set, the wire and envelope formats, the sealed command path with its guarded command set, bounded zone band, anti-replay window, and authenticated state tag, the worm design and its propagation, the blue-half controls (sealed command path, arm authorization, propagation gate, sector erasure, fail-safe policy, build integrity), and an honest threat model that names the shared lab key, the open debug port, the deliberately inert worm, and the limit that it spreads only across the student’s own breadboard nodes as explicit decisions rather than accidents. A 141-case, 458-check native suite reaches 100% line coverage of every owned firmware module.

Index Terms: RP2350, industrial tamper, mesh worm, propagation, reserved flash sector, XChaCha20-Poly1305, Argon2id, anti-replay, authenticated state, malware analysis, infection marker, raw-frame listener, anti-debug, CoreDebug DHCSR, fail-safe, embedded firmware.

I. Introduction

Act I of the OPERATION COLD IRON story was the silent lie: a cold-chain monitor that reported minus eighteen degrees while the store warmed. Act II was the door: an access gate that kept its final verdict in plain SRAM while the cryptography around it was correct. Act III was the payload that is already inside: a valve

controller carrying a benign implant that beacons, arms a logic bomb, and hides from a probe. Act IV was the payload that refuses to die: an HVAC node whose implant kept a copy of itself in a reserved flash sector and re-installed on every boot. Act V is the payload that spreads. The valve led to the pipeline, the pipeline led to the plant, the plant led to the air, and the air led to

the cabinets. The next device does not just survive its removal; it teaches its neighbors to survive theirs.

The industrial tamper system is that next device, and it is a ring. Each node watches a cabinet, reads the cabinet temperature, compares the space to an authorized verdict, drives a shutter latch, and reports its own health on an LCD with total confidence. The frame pulled from the HVAC node carried a route, and the route ended at a NorthPharma cabinet ring. The thing that copies back is already running in the ring, and it has learned to hand a frame to the next node. NorthPharma is the Ministry's front; FROSTLINE wrote the web.

The reveal that ties the five acts together is the shift in what remediation means. Act I was a lie about a number. Act II was a lie about a person. Act III was a lie about machinery, because a second party shared the chip. Act IV was a lie about the recovery, because the payload kept state the image did not own. Act V is a lie about containment, because the payload keeps a copy outside the board. The five-act arc is a widening of the trust boundary: first the sensor, then the state, then the firmware image, then the removal procedure, and now the network itself.

A tamper node is a simple machine. A cabinet sensor reports the space, a gateway authorizes a command, an actuator moves a latch, and an annunciator says whether the node is secure. Three properties must hold at once: integrity, so the command that reaches the latch is the authorized one; authority, so a local input cannot bypass the decision; and state, so the controller does not trust a stale or tampered verdict. The naive controller collapses all three. Act V both fixes that and then goes further, because the propagation act has to answer a question a protocol cannot: what happens when one infected node can make another?

The classroom goal is to teach both halves. The red half and the malware track find the defects: forge a command, replay a captured command, read the reserved sector, break the propagation, clear the infection marker, inject the magic on the raw frame, and step past the anti-debug trap. The blue half and the fix track seal the ring: a sealed and guarded tamper command path, a monotonic anti-replay window, an authenticated state tag, an arm request that asks for authorization instead of bypassing it, a fail-safe policy, a closed propagation gate, sector erasure, and build-level integrity. The centerpiece is a lesson about scope: the wire is authenticated, the

verdict is tagged, and the worm still propagates, because the worm never needed the wire and it listens underneath it.

I.I. Contributions

This paper provides the following concrete contributions:

- A bare-metal RP2350 industrial tamper system that drives the full Embedded Hacking peripheral set: an SG90 shutter latch actuator, a VS1838B NEC arm/disarm remote with ARM, DISARM, and CLEAR commands, a DHT11 cabinet temperature sensor, a 1602 LCD tamper readout over I2C, a red/yellow/green annunciator, a debounced manual arm request, and RYLR998 command and mesh traffic with a declared-length payload parser.
- A sealed tamper command path with a guarded command set, a bounded zone band, a monotonic anti-replay sequence window, and an authenticated state tag that detects a debugger-written verdict before the latch moves.
- An in-repo, third-party-free cryptographic layer: Argon2id key derivation (RFC 9106) and XChaCha20-Poly1305 authenticated encryption (RFC 8439 with an HChaCha20 subkey), sealed per frame into a lowercase hex envelope with the tamper node identifier bound as associated data.
- A benign FROSTLINE worm, confined to a SANDBOX_ONLY build, that demonstrates a raw-frame magic listener, a reserved-sector infection marker, re-install on boot, an autonomous propagation loop, and a CoreDebug DHCSR anti-debug trap.
- A mesh gateway that authenticates before it parses, logs authenticated and rejected requests distinctly, and answers only authenticated requests with a sealed command, plus a spoofing client whose forged and replayed commands are rejected.
- A corpus-aligned packet artifact contract (packet_artifact.json / packet_artifact.h) with a build-time staleness guardrail.
- A 141-case, 458-check native test suite reaching 100% line coverage of every owned firmware module, including the worm, propagation, and infection paths, checked against published RFC test vectors.
- A threat model that states explicitly what the lab profile does and does not protect, and an honest ac-

count of the worm as a sanitized educational artifact confined to the student’s own breadboard nodes.

II. Related Work

Industrial and physical-security monitoring is a mature field, and the cabinet ring is a canonical target. The DHT11 one-wire sensor [1] and the NEC infrared remote encoding [2] are broadly documented and representative of the climate and operator surfaces real installations deploy. The authenticated construction we use follows the ChaCha20-Poly1305 standard [6], and Argon2 follows the Argon2 specification [7]. The stateful construction is the standard replay defense found in secure-messaging and payment protocols, applied here at the scale of one latch.

Three lines of work frame Act V. The first is the long line of firmware implants and logic bombs [8]: code that lives on the device, waits for a trigger, and acts through the device’s own actuators rather than through its protocol. The second is persistence and anti-analysis: a payload that stores a copy of itself outside the code it is trying to survive, and that inspects the system to decide whether it is being observed. The third is propagation: a payload that treats every peer as a new host, so the unit of infection is the network and not the board. The reserved-sector marker, the raw-frame magic listener, and the CoreDebug DHCSR check used here are minimal, well-known examples of those techniques, chosen because they are legible on a debug probe and cheap to verify.

The pedagogical use of intentionally vulnerable firmware is established [5]. The difference in this act is that the artifact is not only a hostile module that coexists with a correct one, and not only a hostile module that survives the removal procedure; it is a hostile module that survives the network. The exercise demonstrates the propagation, the marker, the trap, and the complete removal on the same board, and it makes the scope limit explicit: an authenticated link does not authenticate the machine under it, a reflash does not reach state outside the program region, and a clean board does not clean the ring.

III. System Model

The system consists of four roles:

- **Tamper node (RP2350 firmware):** decodes the infrared arm/disarm remote, verifies and authorizes sealed gateway tamper commands, announces INTRUSION, checks the DHT11 cabinet band, drives the servo latch, handles the manual arm request, renders the tamper readout, and (SANDBOX_ONLY) runs the worm.
- **Mesh gateway (gateway):** listens on the instructor serial port, authenticates and logs every +RCV frame to `tamper_log.csv`, decides authorization, and answers an authenticated request with a sealed tamper command carrying a monotonic sequence number and an authenticated state tag.
- **Edge simulator:** a laptop process that behaves like an additional node, sealing zone requests with the same field key.
- **Attacker:** a laptop process that claims the gateway address, forges a command, or replays a captured command at the node.

Let $A \in \{0,1\}^{\{16\}}$ be the LoRa node address, L the declared payload byte length, and C the ASCII payload, which is a lowercase hex envelope. The unauthenticated wire framing is:

+RCV=, A, ,, L, ,, C, ,, rssi, ,, snr, CRLF

Because the hex payload has no commas, the framing is simpler than Act I’s comma-bearing JSON, but the receiver still slices by declared length rather than by counting delimiters, for exactly the reason Act I documents.

III.I. Hardware Configuration

The classroom node is a Pico 2 (RP2350) carrying the full Embedded Hacking kit. The pin map is identical to Acts I to IV so one breadboard serves all five, and it is fixed in `include/tamper_sys.h` and enforced by the native test suite:

Signal	RP2350 GPIO
DHT11 cabinet temperature sensor (one-wire)	GP4
1602 LCD SDA (I2C1)	GP2
1602 LCD SCL (I2C1)	GP3
RYLR998 RX (UART1 TX)	GP8
RYLR998 TX (UART1 RX)	GP9

Infrared arm/disarm remote (VS1838B)	GP5
Shutter latch actuator (SG90 PWM)	GP14
Manual arm/disarm button	GP15
Red INTRUSION LED	GP16
Yellow ARMED LED	GP17
Green SECURE LED	GP18
Onboard heartbeat LED	GP25

The LCD backpack uses the PCF8574 at 7-bit address 0x27. The servo runs from a 50 Hz PWM output with a 1000 uF bulk capacitor on the 5 V rail to absorb the stall current when the latch moves; seated is 0 degrees and open is 90 degrees. At boot the node programs its own transceiver (AT+ADDRESS=7, AT+NETWORKID=18) and the gateway programs the receiver (AT+ADDRESS=1, AT+NETWORKID=18) before logging, so command traffic is only delivered between radios that share the network identifier.

The latch is fail-safe: it is driven to the seated position at initialization and on every failure path, so loss of power, a failed cabinet read, a malformed command, a tampered verdict, or a lost link all leave the latch closed and the zone returned to the fail-safe value (0). The manual arm request is a request, not an authorization, and it never moves the latch on its own.

III.II. Operator Remote, Cabinet Sensor, and Annunciation

The VS1838B is a 38 kHz demodulating infrared receiver whose output idles high and pulls low during a mark. The decoder times edges and reconstructs a NEC pulse train, then feeds the command into the request set. MONITOR_IR_ARM is 0x47, MONITOR_IR_DISARM is 0x45, and MONITOR_IR_CLEAR is 0x46. The optical surface has no key and no challenge, so an arm command is treated as a request, not as an authorization; the sealed radio path is what moves the latch in the defended design, and the optical path is a surface the red half examines.

The DHT11 is the cabinet temperature sensor. A reading that fails its checksum is never safe, and a valid reading outside the band (TAMPER_CLIMATE_MIN_TENTHS = 0 to TAMPER_CLIMATE_MAX_TENTHS = 400, that is 0.0 C to 40.0 C) is out of band. Either case marks the space as not

nominal, so a dead or unplugged sensor, or a genuinely unsafe cabinet, is visible in the tamper readout.

Exactly one status lamp is lit at a time. Red is INTRUSION, yellow is ARMED while the node is armed or an arm request awaits authorization, and green is SECURE. The 1602 LCD shows the latch state and the link on line one (ST:SECURE L:UP) and the zone and the infection status on line two (ZN:0 I:--).

IV. Wire Protocol

The arm control or the edge simulator seals a two-byte zone into an XChaCha20-Poly1305 envelope and sends it to the gateway:

```
AT+SEND=0001,84,<84 lowercase hex characters>
```

The gateway answers an authenticated request with a sealed tamper command. The command plaintext is a 23-byte body:

```
seq[4] (little-endian) || command[1] || zone[2]
(little-endian) || tag[16]
```

where seq is the monotonic gateway sequence number, command is one of the guarded tamper commands TAMPER_COMMAND_ALERT (0x01), TAMPER_COMMAND_ARM (0x02), or TAMPER_COMMAND_SECURE (0x03), zone is the authorized zone in the 0 to 16 band, and tag is a tag over the authorization record the command would produce. The gateway sends the reply back to the claimed sender:

```
AT+SEND=<node>,126,<126 lowercase hex characters>
```

The radio's AT command buffer (RADIO_AT_CMD_MAX_LEN), the inbound +RCV buffer (RADIO_RCV_MAX_LEN), and the generated artifact limit (PACKET_MAX_RCV_LEN) are all 256 bytes, which comfortably holds the largest possible envelope plus framing. The line accumulator is one byte larger than the command limit so it can hold the terminating NUL.

The firmware enforces a guarded command set and a bounded zone band in control_parse: the recovered command byte must be one of the three guarded tamper codes, and the recovered zone must lie between TAMPER_ZONE_MIN (0) and TAMPER_ZONE_MAX (16). This is the sealed replacement for the unauthenticated tamper injection, and it means a raw value or an out-of-band zone can never reach the actuator decision.

IV.I. Envelope on the Wire

The sealed envelope is the lowercase hexadecimal encoding of a fixed layout:

```
nonce[24] || ciphertext[L] || tag[16]
```

For a two-byte request body this is $24 + 2 + 16 = 42$ bytes, or 84 hex characters. For a 23-byte command body this is $24 + 23 + 16 = 63$ bytes, or 126 hex characters. The maximum plaintext is 48 bytes (`ENVELOPE_MAX_PLAINTEXT`), so the largest possible envelope is $24 + 48 + 16 = 88$ bytes, or 176 hex characters plus a trailing NUL, for a 177-byte envelope buffer (`ENVELOPE_MAX_HEX_LEN`). The declared length L in the framing is the length of the hex string, not of the underlying plaintext.

IV.II. Declared-Length Slicing Invariant

Given the substring T after the second comma:

$$C = T[0 : L] \quad \text{and} \quad T[L] = ,$$

The invariant $T[L] = ,$ is checked, so a mismatch between the declared length and the actual payload is a parse error rather than silent corruption. This is the same discipline Act I adopts for comma-bearing JSON, retained here for uniformity and for defense against a hostile declared length.

V. Cryptographic Design

The radio is the first open path, and it is the one a key can close; the worm is the second open path, and it is one a key cannot close. The design goal is that a forged or modified frame must fail before any decision is made, while acknowledging that a payload which never presents a frame to the authenticated path is unaffected by that path. Two primitives provide the first property, and both are implemented in this repository with no third-party code.

V.I. Argon2id Key Derivation

A passphrase is not a key. Argon2id (RFC 9106) [7] is a memory-hard password hash that mixes the passphrase and a salt across memory and time so that recovering the field passphrase from a captured image is expensive. The node derives a 32-byte key at initial-

ization with the classroom profile $t=3, p=1, m=64$ blocks (`CRYPTO_KDF_TIME_COST`, `CRYPTO_KDF_PARALLELISM`, `CRYPTO_KDF_MEMORY_BLOCKS`). That profile is sized to fit the RP2350 SRAM budget; it is a teaching parameter, not a hardening parameter, and the documentation says so. The salt must be at least 8 bytes; the laboratory salt is the 16 ASCII bytes `coldiron-salt-01`. The in-repo derivation is built from BLAKE2b and the Argon2 variable-length hash H' , and the RFC 9106 known-answer test runs in the Python suite.

V.II. XChaCha20-Poly1305 per Frame

Every frame is sealed with XChaCha20-Poly1305, an AEAD that combines the ChaCha20 stream cipher and the Poly1305 one-time authenticator from RFC 8439 [6] with an extended-nonce construction. The 24-byte nonce is expanded through HChaCha20 into a per-frame subkey, which yields two properties that matter here:

- **Unpredictable nonces at scale.** A 192-bit nonce may be drawn at random for every frame from the RP2350 hardware random source, so the node never needs a shared counter that a reboot could reuse.
- **One pass for secrecy and integrity.** The same operation produces the ciphertext and a 128-bit Poly1305 tag. An attacker who guesses a valid tag succeeds with probability 2^{-128} .

The associated data is the tamper node identifier, a single byte (0x07 for the default node). It is authenticated but not encrypted, so a frame sealed for one node cannot be silently relabeled as another node's frame.

V.III. Why ChaCha20 over AES on the RP2350

The RP2350 does not have a hardware AES engine; its accelerated crypto block covers SHA-256, not AES. A software AES implementation on this part is therefore both slower and riskier: table-driven AES performs data-dependent memory accesses, and those accesses create a cache-timing side channel. ChaCha20 is built only from addition, rotation, and XOR, with no data-dependent table lookups, so it is fast in portable C and has no comparable cache-timing surface. XChaCha20-Poly1305 is thus both the modern choice and the pragmatic one for this silicon.

The primitives are split across small, independently testable modules: `src/chacha20.c`, `src/poly1305.c`, `src/crypto_aead.c`, `src/blake2b.c`, `src/argon2.c`, `src/crypto_kdf.c`, and `src/envelope.c`. A constant-time comparison (`crypto_aead_tag_equal`) ensures a mismatching tag is rejected without an early-exit timing signal.

V.IV. Key Model

Act V uses a single field key. It seals every frame on the wire and it computes the state tag over the authorization record. In the classroom build the field key is derived from one committed lab passphrase and salt, so the firmware and the gateway interoperate with no provisioning step. That is a lab convenience, not a deployment, and the documentation says so. The design keeps the roles separable so a student can reason about the real lifecycle: derive, provision per device, use, rotate on a schedule, and retire. A production build provisions key material from one-time-programmable (OTP) memory and keeps the state-tag key off the field device where possible. The worm is deliberately orthogonal: it is never given a key, it never opens an envelope, and it demonstrates that a valid key does not stop an adversary who never needs one.

VI. Anti-Replay Window and Authenticated State

Strong AEAD is necessary and not sufficient. Two stateful controls sit above the sealed wire.

VI.I. The Anti-Replay Window

A captured command is authentically sealed, so a controller that checks only the tag will happily apply it again. The authorization record keeps `last_seq`, the highest sequence number ever accepted, and `tamper_auth_apply` accepts a command only when its sequence is strictly greater than `last_seq`. The order of checks is deliberate: the sequence test is evaluated first, then the tag is verified against the candidate record the command would produce, and only then is the record updated. A replayed valid command therefore fails on freshness, not on cryptography, which is exactly the lesson: authentication is not freshness.

VI.II. The Authenticated State Tag

The centerpiece is the verdict itself. The controller decides with a boolean in SRAM, call it `granted`, and an attacker with a debug probe and a GDB session does not break the cipher; they set `granted = true`. To detect that, the authorization record is nine bytes:

```
record = granted[1], seq[4], last_seq[4]
```

and the state tag is an XChaCha20-Poly1305 tag over that record, computed under the field key with a deterministic nonce built from the sequence number and the domain byte `0xA7`:

```
tag = AEADseal(fieldkey, nonce(seq), record, AD = ∅)
```

`tamper_auth_state_ok` recomputes the tag and compares it in constant time, and the guarded command path requires it before the latch moves. A debugger that flips `granted` without recomputing the tag changes the record, so the stored tag no longer matches and the release is denied ahead of the actuator. The wire is authenticated, and so is the verdict.

VI.III. TOCTOU in One Session

The two attacks are independent and teach different defaults. The window stops a valid command from working twice. The tag stops an unauthorized verdict from existing at all. Together they convert the original failure, a correct decision followed by a mutable state read (a time-of-check to time-of-use gap), into two explicit, testable checks.

VII. Envelope Layout and Gateway Verification

The binary envelope is assembled in a fixed order and then hex-encoded:

```
envelope = nonce[24], ciphertext[L], tag[16]
```

The encoder emits lowercase hex with a trailing NUL, and the decoder accepts either case. It requires an even-length string of at least the nonce plus tag size, bounds the decoded length, recomputes the tag over the associated data and ciphertext, compares in constant time, and only then decrypts. Any malformed, truncated, tampered, or forged envelope returns false and yields no trusted plaintext.

On the gateway side, `scripts/gateway.py` mirrors the same construction in pure Python using the standard library and the `field_crypto` module. The processing order is deliberate:

1. Parse the +RCV line by declared length to recover the hex envelope.
2. Authenticate and open the envelope. If the tag does not verify, log the frame as UNAUTHENTICATED with an empty zone and stop. The forged body is never parsed.
3. Only for an authenticated frame, recover the two-byte zone, check it against the bounded band, write an OK row, and answer with a sealed command carrying the next monotonic sequence and the state tag over the record the command would produce.

The CSV log therefore grows by one row per frame with columns `utc`, `sender`, `auth`, `zone`, `rsi_snr`, and the `auth` column is the audit trail. The spoofing client `scripts/spoof.py` holds no field key, so it cannot produce a valid envelope, and in replay mode it can only resend a captured command that the window will refuse.

VIII. The FROSTLINE Worm: Propagation and Infection

Act V is the propagation act. The worm is real in technique and inert in effect, and it is confined to a single module compiled only under a build guard. This section states what it does, how it spreads, how it is detected, and the honest limit of the artifact.

VIII.I. Build Guard and Safety Boundary

`src/implant.c` is compiled only when `SANDBOX_ONLY` is defined. The clean firmware build does not define it, so the shipping image contains no worm. The native test build and the companion CTF build do define it, and the test build also defines `IMPLANT_HOST MOCK`, which replaces the CoreDebug register and the reserved flash sector with controllable host variables. This is the containment boundary: the malware track is a build configuration, not a hidden runtime feature of the shipping firmware. There is no network, no filesystem, and no host impact; the reserved sector is on the same chip and holds nothing else, and the only effect of the worm

is on the student's own radio traffic and the infection field on the student's own LCD.

VIII.II. Reserved-Sector Infection and Re-Install on Boot

`implant_init` reads the marker byte at `TAMPER_IMPLANT_RESERVE_ADDR` (`0x103FF000`), the final sector of external flash. On the first run the marker is absent, so the implant erases the sector and programs `0xC7` (`TAMPER_IMPLANT_MARKER_BYTE`) with the Pico SDK flash API once. On every later boot the marker is present, so the worm re-arms its payload handler without any firmware change. A reflash that rewrites the program region does not touch the reserved sector, so the payload survives the procedure that was supposed to remove it. That is the persistence half of the lesson, and it is why neutralization requires both the erasure of the reserved sector and the removal of the re-install code path.

VIII.III. The IRONWEB Frame and the Propagation Loop

The worm magic is the 7-byte preamble `IRONWEB` (`TAMPER_IMPLANT_WORM_MAGIC`). One frame is 11 bytes (`TAMPER_IMPLANT_WORM_LEN`): the magic, then a 4-byte synthetic status body carrying the marker byte `0xC7`, the low and high bytes of the tick counter, and a byte that reports whether a probe is attached. The frame carries no real data and targets no external address.

`implant_tick` advances a monotonic tick counter. While the node is infected and no probe is attached, every `TAMPER_IMPLANT_PROPAGATE_INTERVAL_TICKS` (4) ticks the worm builds the `IRONWEB` frame and emits it over the LoRa mesh link. A peer that receives the frame and runs the same handler matches the magic, infects itself, and emits the frame again. The infection therefore walks the ring one hop at a time, and the unit of infection is the network rather than the board.

VIII.IV. The Raw-Frame Magic Handler

The decisive detail is where the worm listens. `monitor_apply_frame` delivers the raw inbound payload to `implant_handle_command` before it calls the

sealed control_handle_frame. The worm therefore matches the 7-byte magic on the raw payload, underneath the envelope, and it never presents a sealed frame of its own. A forged, unauthenticated, or replay-protected command is irrelevant to a payload that reads the bytes before authentication. That is the scope lesson of the act: a correct sealed command path does not protect a listener that sits below it.

VIII.V. The Anti-Debug Trap

Every tick, `implant_tick` calls `implant_debug_attached`, which reads the CoreDebug DHCSR register at `0xE000EDF0` (`TAMPER_IMPLANT_DHCSR_ADDR`). Bit 0 is `C_DEBUGEN` and bit 1 is `C_HALT` (`TAMPER_IMPLANT_DHCSR_DEBUGEN` and `TAMPER_IMPLANT_DHCSR_HALT`). When either bit is set, the worm returns early, so both the magic handler and the propagation are suppressed while a probe is attached. The worm behaves benignly under observation and resumes when the probe is detached. This is the minimal anti-analysis trap, and it is deliberately simple so a student can see the branch, set a breakpoint after it, and prove the payload.

VIII.VI. Detection and Neutralization

The worm is detected by image comparison: the clean build and the `SANDBOX_ONLY` build differ by the implant module and its symbols. It is detected by the reserved-sector marker at `0x103FF000`, which is state the firmware image does not own. It is detected on the mesh by the periodic 11-byte IRONWEB frame and its four-tick cadence, and on the LCD by the `I:INF` infection field. It is detected by static analysis by the IRONWEB magic, the `implant_propagate` emission, and the DHCSR read address. It is detected under GDB because the DHCSR read is a branch that a student can stand after. The native implant tests (`test_monitor_implant_frame`, `test_implant_init_first_run`, `test_implant_reinstall_on_boot`, `test_implant_infected_marker`, `test_implant_debug_attached`, `test_implant_propagation_flag`, `test_implant_worm_match`, `test_implant_build_worm`, `test_implant_propagate_gate`, `test_implant_handle_command`,

`test_implant_propagates`, `test_implant_anti_debug`, and `test_implant_tick_clean`) assert each behavior and its containment.

Neutralization is not a one-byte patch. It is the closure of the propagation gate, the erasure of the reserved sector, the removal of the code path, and the removal of the build flag, plus image signing and a debug lockdown on a deployed part. In the lab, the anti-debug trap is defeated by understanding the branch, not by hiding from it.

VIII.VII. Honest Limitation

The worm is a benign educational worm. It is confined to the breadboard, guarded by `SANDBOX_ONLY`, and has no network. It transmits only over the student's own LoRa modules on the classroom network id, it actuates nothing, and it writes only to a reserved sector on the same chip that holds nothing else. It is a demonstration of technique, not tradecraft: it does not encrypt itself, it does not load a second stage, it does not exfiltrate anything, and it does not resist a determined physical attacker who can erase the sector. It spreads only across the student's own breadboard nodes, and its propagation is a re-emission over the same classroom radio link, not an internet worm. Its persistence is persistence against a firmware reflash, and its propagation is propagation against a network the student owns. The value of the exercise is that it makes the scope limit of a firmware image concrete: reflashing is not remediation when the payload keeps state the image does not own, and cleaning one board is not remediation when the payload keeps a copy on the wire.

IX. Artifact Contract

Provisioning constants are stored in a JSON artifact:

```
{
  "format": "industrial-tamper-packets-demo-v1",
  "frame_version": 1,
  "node_address": 7,
  "gateway_address_hex": "0001",
  "frame_size": 48,
  "alert_wait_ms": 5000,
  "latch_close_pulse_us": 500,
  "latch_open_pulse_us": 1500,
  "dht_timeout_us": 240,
  "lcd_i2c_address_hex": "27",
  "max_rcv_len": 256,
  "example_frame": "{ \"cmd\": \"alert\"",
```

```
\\"zone\\":1}"
}
```

`scripts/gen_packet.py` emits `include/packet_artifact.h` from the JSON byte-for-byte. The CMake build regenerates the header before compiling and fails when the committed header is stale, so firmware constants and the test suite always read the same provisioning data. The receive limit is 256 bytes, matching the radio command and receive buffers so a maximum-size hex envelope fits with framing headroom.

X. Fail-Safe Policy and Arm Authorization

A latch has a safe state, and the controller must choose it deliberately. The latch is **fail-safe**: `latch_init` seats it closed at boot, `latch_fail_safe` drives it closed and latches a fault, and the monitor calls `monitor_fail_safe` on link loss, which also returns the fail-safe zone (0). A lost gateway link for longer than `TAMPER_ALERT_WAIT_MS` leaves the latch closed and the safe zone active, because a command that cannot be authorized must not be assumed.

The manual arm request is a local request, and it must not silently bypass authorization. `monitor_handle_arm` and `monitor_apply_ir_command` raise `g_request_pending`; they never move the latch on their own. `monitor_apply_command` clears the pending indication only when an authorized command arrives. A one-button request therefore cannot outrank a sealed, authorized command. The lesson is that fail mode, priority, and the difference between a request and an authorization are policy choices, and naming them is part of the design.

XI. Attack Exercises and Hardening

The classroom runs the malware track and the fix track against the same build.

XI.I. Malware Track: Propagation, Marker, Magic, and Trap

Students flash the `SANDBOX_ONLY` image, read the `0xC7` marker in the reserved sector `0x103FF000`, and prove

they can erase it. They watch the mesh and locate the 11-byte `IRONWEB` frame and its four-tick cadence, then close the propagation gate and prove the node stops emitting it. They reflash the firmware, boot again, and, with a marker present, watch the worm re-arm. They then inject the `IRONWEB` magic on the raw payload and observe the node infect itself and emit the frame again, with no sealed envelope and no field key. Finally they attach a probe, observe that the worm suppresses itself, break after the `DHCSR` check, and prove the payload with the trap bypassed. The centerpiece is the scope claim in one session: a correct, sealed command path does not protect a listener that reads the raw bytes underneath it, and cleaning one node does not clean the ring.

XI.II. Red Half: Forgery and Replay

A structurally plausible command with a random nonce and a random tag is injected with `scripts/spoof.py --mode bad-tag`. The spoof tool holds no field key, so the tag cannot verify, and the controller denies before parsing any body. A captured command is replayed with `--mode replay`; the sequence is not greater than `last_seq`, the command fails on freshness, and the intrusion lamp lights. This is a pedagogical reintroduction of a well-known link failure mode: at the physical and MAC layer nothing binds a frame to a physical transceiver, so authentication must live in the payload.

XI.III. Red Half: The Verdict in SRAM

The exercise halts the controller under the Debug Probe, breaks in the authorization path, sets `granted = true`, and continues. Under a controller that trusts the boolean the latch moves. Under the Act V controller the state tag is recomputed over the modified record, the mismatch is found, and the command is denied ahead of the actuator.

XI.IV. Blue Half: Sealing It

The blue-half controls map one-to-one onto the red-half and malware findings:

- **Sealed command path.** Open the envelope under the field key, guard the command byte against the tamper set, bound the zone to the provisioning band, verify the sequence and the state tag.
- **Anti-replay window.** `last_seq` and a strictly monotonic sequence rule.

- **Authenticated state tag.** A keyed tag over the nine-byte authorization record, computed with a domain-separated nonce and verified in constant time.
- **Arm authorization.** A local request raises the pending indication and never bypasses authorization.
- **No re-broadcast.** The production firmware never relays an untrusted frame; the propagation gate stays closed and the worm path is compiled out.
- **Fail safe.** Seat the latch at boot, on link loss, and on every fault, and return the fail-safe zone.
- **Propagation removal.** Close the gate, erase the reserved sector, and remove the re-install code path together, because either one alone is insufficient.
- **Build integrity.** Do not define `SANDBOX_ONLY` in production, and sign and verify the firmware image.
- **Debug lockdown.** On the deployed part, burn secure-boot and debug-disable in OTP so SWD cannot read or write SRAM.

XI.V. What the Hardening Buys, and What It Does Not

The command path closes the forgery, replay, and verdict-tamper surfaces:

- A forged or modified frame fails the tag before parsing.
- A captured valid command fails on freshness on second use.
- A debugger-written verdict fails the state-tag check before the latch moves.
- The node identity is bound into the associated data, so a frame cannot be relabeled for another node.

It does not, by itself, stop a payload that listens below the authenticated path, and it does not remove the copy of that payload in the reserved sector or the copy on a neighbor. That is a build-integrity, network-containment, and state-erasure problem, not a protocol problem, and it is the central lesson of the act.

XII. Implementation Compliance Mapping

The repository implements the full classroom loop:

- **Peripherals and control:** `src/monitor.c` drives the tick and the latch policy; `src/latch.c` sequences the actuator and fails safe; `src/sensor.c` samples the DHT11 cabinet band; `src/display.c` renders

the tamper readout; `src/status_led.c` maps the verdict to the red, yellow, and green lamps; `src/button.c` debounces the manual arm request; `src/servo.c` drives the latch PWM; `src/ir_remote.c` decodes NEC arm/disarm commands.

- **Command and state:** `src/control.c` opens and applies sealed commands with a guarded command set and a bounded zone band; `src/tamper_auth.c` holds the authorization record, the monotonic anti-replay window, and the authenticated state tag.
- **Malware:** `src/implant.c` implements the `SANDBOX_ONLY` IRONWEB magic listener, the reserved-sector infection marker, re-install on boot, the propagation loop, and the CoreDebug anti-debug trap.
- **Radio:** `src/radio.c` provisions the transceiver, builds AT+SEND, parses +RCV with the declared-length discipline, and pumps CRLF lines into 256-byte buffers.
- **Cryptography:** `src/chacha20.c`, `src/poly1305.c`, `src/crypto_aead.c`, `src/blake2b.c`, `src/argon2.c`, `src/crypto_kdf.c`, and `src/envelope.c`, with `include/field_secrets.h` holding the lab-only key material.
- **Tooling:** `scripts/gen_packet.py`, `run_tests.py`, `check_coverage.py`, `audit_c_standard.py`, `audit_python_standard.py`, and `gen_banner.py`.
- **Classroom:** `scripts/gateway.py` (gateway provisioning, authentication, CSV logging, sealed command replies), `scripts/spoof.py`, `scripts/sim_edge.py`, and the pure-Python interoperable crypto in `scripts/field_crypto.py`.
- **Tests:** 141 native C cases and 458 checks with 0 failures. They cover the full DHT waveform and every timeout shape, the latch state machine and its bounded travel, the sealed command path and its guards, the authorization window and state tag, the arm no-bypass path, fail-safe on link loss, the declared-length parser, the servo and LED mappings, the arm/disarm remote paths, the worm first run, re-install on boot, the IRONWEB magic, frame construction and propagation gating, anti-debug, and write-once infection, and the cryptographic primitives against published vectors.

The tests run natively on the host via mock Pico SDK headers, reaching 100% line coverage on `crc.c`, `sensor.c`, `display.c`, `radio.c`, `status_led.c`, `button.c`, `servo.c`, `ir_remote.c`, `latch.c`, `control.c`, `tamper_auth.c`, `chacha20.c`,

poly1305.c, crypto_aead.c, blake2b.c, argon2.c, crypto_kdf.c, envelope.c, monitor.c, and implant.c under LLVM source coverage, for 2052 / 2052 lines. main.c is excluded from coverage by design.

XIII. Threat Model and Limitations

The security claims of this build are bounded and stated plainly.

- **Lab key profile.** Argon2id runs at $t=3$, $p=1$, $m=64$ blocks so the derivation fits the RP2350 SRAM budget. This is weaker than a production password-hashing profile and must be raised on a host gateway.
- **Keys in flash are development-only.** `include/field_secrets.h` commits a shared passphrase and salt so the firmware and the Python gateway derive the same key in the classroom. Production firmware must provision key material from OTP memory at manufacture and must never embed a passphrase, salt, or derived key in flash.
- **Open debug port.** The Debug Probe is the instrument for both the malware analysis and the verdict-tamper exercise. The authenticated state tag makes a tampered verdict detectable, but a probe that can read the field key from SRAM defeats the design. Production must disable debug in OTP.
- **Replay scope.** The window rejects a replayed command, but a reboot resets `last_seq` to zero. A command captured before a reboot can therefore be replayed after one. A production controller persists the sequence floor in non-volatile memory.
- **The worm is benign, guarded, and breadboard-bound.** It is compiled only under `SANDBOX_ONLY`, confined to the breadboard, and has no network. It touches only its own radio frame, its arming latch, and the reserved sector on the same chip. It does not survive a deliberate sector erase and does not resist physical forensics.
- **Propagation is classroom-scoped.** The worm spreads only across the student's own breadboard nodes on the classroom network id, and its propagation is a re-emission over the classroom radio link. It is not an internet worm and it has no external address. The documented neutralization is the closure of the propagation gate plus the sector erase plus the code removal.

- **The worm listens below the protocol entirely.** No amount of wire authentication stops a module that reads the raw payload before the envelope is opened. Mitigating that is a build-integrity, signing, and debug-lockdown problem, not a protocol problem.
- **Infection is bounded.** The reserved sector is on the same chip and holds nothing else, and the propagation target is the classroom mesh, so the persistence and the propagation are controlled demonstrations, not an operational implant.
- **Fail mode trade-off.** The latch is fail-safe and the manual arm request cannot bypass authorization. Any change to either must be a policy decision, not a code accident.
- **Infrared path unauthenticated.** The NEC arm/disarm remote has no key and no anti-replay state. Any compatible remote can send a request. The sealed radio path is the authorization path; the optical surface is a documented exposure.
- **Sensor trust boundary.** The DHT11 is a checksummed but not authenticated one-wire sensor; the cabinet band is only as trustworthy as the physical wiring and the edge timing.
- **RSSI and SNR are informational.** Neither is a reliable origin indicator.
- **Artifact guardrail.** The build-time artifact check verifies provisioning consistency, not security.
- **Denial of service.** An attacker on the band can still jam or flood the receiver; authentication is not availability.

XIII.I. Future Work

- Provision the field key from RP2350 OTP memory and add a documented rotation procedure.
- Persist the anti-replay sequence floor in non-volatile memory so a reboot does not reset freshness.
- Add a signed-image verification step to the flash procedure and a measured boot chain on the RP2350.
- Add a reserved-sector erasure step to the documented flash procedure so the propagation lesson maps to a repeatable remediation.
- Add a network-level quarantine and key-rotation procedure so one infected node can be isolated and re-keyed without touching every peer.
- Burn debug-disable and secure-boot settings in OTP for the deployed part.

- Add an actuator-state ledger and an interlock alarm debounce.
- Raise the Argon2id profile on the gateway and record the derivation cost as a measured parameter.
- Extend the malware track toward the Act VI exfiltration lesson with a controlled covert channel and its containment procedure.

XIV. Conclusion

OPERATION IRON WEB turns a trusting tamper ring into a defensible one, and then shows why a defended protocol is not the whole story. The node drives the full Embedded Hacking peripheral set, so a state failure has a visible and physical consequence at the shutter latch. The LoRa command path is sealed end to end with XChaCha20-Poly1305 keyed through Argon2id, implemented and tested entirely in-repo, with the tamper node identity bound as associated data. Act V adds a guarded command set and a bounded zone band, a monotonic anti-replay window so a captured command dies on second use, an authenticated state tag so a debugger-written verdict dies before the latch moves, a manual arm request that asks for authorization instead of bypassing it, and a fail-safe posture that returns the safe zone. The gateway authenticates before it parses, so the spoofing client that once forged a command now fails at the tag, and the replay that once reopened a cabinet now fails at the window. And then there is the worm: a benign, `SANDBOX_ONLY` FROSTLINE module that listens on the raw payload for the IRONWEB magic, writes a marker into a reserved flash sector, re-installs on every boot, and emits its 11-byte frame to the mesh so a neighbor infects itself and emits it again, all without ever touching the sealed wire. The firmware, gateway toolset, artifact guardrail, and 100%-line-covered native test suite provide a reproducible baseline, and the threat model states exactly which assumptions remain. That combination, a sealed command path next to an honest account of the module that spreads underneath it, is the lesson Act V owes the story: the protocol was never the hard part. The copy on the next board was.

XV. References

[1] D-Robotics, “DHT11 Digital temperature and humidity sensor datasheet,” Aosong Electronics Co., Ltd, 2010.

- [2] Vishay Semiconductors, “IR Receiver Modules for Remote Control Systems (VS1838B),” Vishay Intertechnology, datasheet 81910, 2018.
- [3] Anonymous the Security Researcher, “Analysis of serial-AT sub-GHz radios: cleartext configuration and absent frame authentication,” Embedded security working notes, 2022.
- [4] R. Menon and A. Prakash, “On the (in)security of LoRa point-to-point links under address spoofing,” *ACM SIGCOMM Embedded Systems Workshop*, 2023, pp. 12-19.
- [5] K. Thomas, “The reverse engineering self-study course,” <https://github.com/mytechnotalent/Reverse-Engineering>, 2026.
- [6] Y. Nir and A. Langley, “ChaCha20 and Poly1305 for IETF Protocols,” RFC 8439, Internet Engineering Task Force, June 2018.
- [7] A. Biryukov, D. Dinu, D. Khovratovich, and S. Josefsson, “Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications,” RFC 9106, Internet Engineering Task Force, September 2021.
- [8] A. Costin and J. Zaddach, “A large-scale analysis of the security of embedded firmwares,” *Proceedings of the 23rd USENIX Security Symposium*, 2014, pp. 95-110.